

Programação Concorrente e Distribuída

Threads

Glêdson Elias
gledson@ci.ufpb.br

1



Threads

- Características
  - Permite múltiplas linhas de execução coexistirem em um único processo
  - Compartilham recursos de um processo
    - Espaço de endereços do processo
  - Cada thread possui seu contador de programa individual, pilha e variáveis locais
  - Comunicação via memória compartilhada
  - Também denominado processo leve (lightweight process)

2

Threads

- Características
  - Em monoprocessadores . . .
    - Processador é compartilhado entre os threads
    - Sistema operacional realiza o escalonamento e a alocação de threads ao processador
    - Threads não são executados simultaneamente
    - Provê suporte a concorrência lógica, baseada na divisão do tempo (time slicing)

Threads

- Características
  - Em multiprocessadores . . .
    - Cada thread pode ser alocado a um processador
    - Sistema operacional realiza o escalonamento e alocação de threads aos processadores
    - Threads podem ser executados simultaneamente
    - Provê suporte a concorrência física

Threads

- Características
  - Executam simultânea e assincronamente
    - Threads podem acessar recursos compartilhadas com outros threads
    - Pode produzir resultados não-determinísticos
  - Requer a adoção de mecanismos explícitos de exclusão mútua e sincronização

Threads

- Threads em Java
  - Java suporta concorrência a nível de linguagem
    - A linguagem C suporta concorrência em bibliotecas de funções
  - Adota um modelo preemptivo de execução
    - Intervalos do processador são alocados aos threads em um modelo round-robin
  - Suporta o conceito de prioridade
  - Implementado na classe Thread e na interface Runnable

Threads

- Threads em Java
  - Todo programa consiste de pelo menos um thread que executa o método `main()`
    - thread principal (main thread)
  - Outros threads internos podem ser criados pela própria JVM
    - Depende de cada implementação de JVM
  - Outros threads a nível de usuário podem ser explicitamente criados pelo programa

Threads em Java

- Classe Thread
  - Deve-se estender a classe `Thread`
  - Sobrepor o método `run()`
    - Método executado quando o thread é iniciado
  - Instanciar um ou vários objetos thread
  - Ativar o método `start()` dos objetos thread para iniciar os threads correspondentes

```
public class HelloWorldThread extends Thread {
public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    HelloWorldThread t = new HelloWorldThread();
    t.start();
  }
}
```

Threads em Java

- Classe Thread

```
class MyThread extends Thread {
  private String message;
  public MyThread(String m) {message = m;}
  public void run() {
    for(int r=0; r<10; r++)
      System.out.println(message);
  }
}
```

```
public class ThreadDemo {
  public static void main(String[] args) {
    MyThread t1,t2;
    t1=new MyThread("Primeiro Thread");
    t2=new MyThread("Segundo Thread");
    t1.start();
    t2.start();
  }
}
```


Threads em Java

- Interface Runnable
  - Deve-se implementar a interface `Runnable`
  - Implementar o método `run()`
    - Método executado quando o thread é iniciado
  - Instanciar um ou vários objetos thread
  - Ativar o método `start()` dos objetos thread para iniciar os threads correspondentes

```
public class HelloWorldRun implements Runnable {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    HelloWorldRun h = new HelloWorldRun();
    Thread t = new Thread(h);
    t.start();
  }
}
```

Threads em Java

- Interface Runnable

```
class MyRun implements Runnable {
  private String message;
  public MyRun(String m) {message = m;}
  public void run() {
    for(int r=0; r<10; r++)
      System.out.println(message);
  }
}
```

```
public class RunDemo {
  public static void main(String[] args) {
    MyRun r1,r2;
    Thread t1, t2;
    r1=new MyRun("Primeiro Thread");
    r2=new MyRun("Segundo Thread");
    t1 = new Thread(r1);
    t2 = new Thread(r2);
    t1.start();
    t2.start();
   }
}
```


Threads em Java

- Thread x Runnable
  - Estender a classe `Thread` impossibilita herança de outra classe
    - Java não suporta herança múltipla
  - Implementar a `interface Runnable` permite a herança de outra classe
    - Java permite as primitivas `extends` e `implements` serem usadas conjuntamente

Threads em Java

- Expressões Lambda

```
class MyLambda {
  private String message;
  public MyLambda(String m) {message = m;}
  public static void lambdaByClass() {
    System.out.println("Ref. ao método de Classe");
  }
  public void lambdaByObject() {
    System.out.println(message);
  }
}
```

```
public class LambdaDemo {
  public static void main(String[] args) {
    Thread t1 = new Thread(() -> { System.out.println("Run"); });
    t1.start();
    MyLambda r = new MyLambda("Ref. ao Método de Objeto");
    Thread t2 = new Thread(r::lambdaByObject);
    t2.start();
    Thread t3 = new Thread(MyLambda::lambdaByClass);
    t3.start();
  }
}
```


Threads em Java

- Terminando Threads
  - Um thread termina quando o método `run()` é concluído
    - Método `run()` retorna normalmente
    - Método `run()` lança exceção não capturada
  - Threads não podem ser reinicializados
    - Invocar o método `start()` mais uma vez gera a exceção `IllegalThreadStateException`
  - Método `isAlive()` pode ser usado para verificar se o thread foi terminado

```
t.start()
...
if t.isAlive()
```

Threads em Java

- Terminando Threads
  - Threads não podem ser reinicializados
    - Invocar o método `start()` mais uma vez gera a exceção `IllegalThreadStateException`

```
public class HelloWorldIllegal extends Thread {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    System.out.println("Iniciando ...");
    HelloWorldIllegal t = new HelloWorldIllegal();
    t.start();
    System.out.println("Reiniciando ...");
    t.start();
    System.out.println("Terminando ...");
  }
}
```


Threads em Java

1

- Terminando Threads
  - Método `isAlive()` pode ser usado para verificar se o thread foi terminado

```
public class HelloWorldAlive extends Thread {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    System.out.println("Iniciando ...");
    HelloWorldAlive t = new HelloWorldAlive();
    t.start();
    while(t.isAlive());
    System.out.println("Terminando ...");
  }
}
```

Threads em Java

- Prioridades
  - Pode ser usada para expressar a importância ou urgência de diferentes threads
  - Políticas de escalonamento de threads não são padronizadas em Java
    - Dependente da implementação da JVM
    - Depende do escalonador do sistema operacional
    - Em geral, preferência é dada a threads de maior prioridade

Threads em Java

- Prioridades
  - Cada thread possui uma prioridade que varia entre `Thread.MIN_PRIORITY` (1) e `Thread.MAX_PRIORITY` (10)
  - Por default, cada novo thread tem a prioridade do thread pai
    - Thread principal associado com o método `main` tem a prioridade `Thread.NORM_PRIORITY` (5)
  - Prioridade de um thread pode ser identificada e modificada com os métodos `getPriority()` e `setPriority()`

Threads em Java

- Prioridades

```
class MyThread extends Thread {
  private String message;
  public MyThread(String m) {message = m;}
  public void run() {
    System.out.println("Iniciando " + message);
    for(int i=0; i<100000; i++)
      for(int j=0; j<100000; j++)
        for(int k=0; k<100000; k++);
    System.out.println("Terminando " + message);
  }
}
```

```
public class ThreadDemo {
  public static void main(String[] args) {
    MyThread t1,t2;
    t1=new MyThread("primeiro thread");
    t2=new MyThread("segundo thread");
    t1.setPriority(1);
    t2.setPriority(10);
    t1.start();
    t2.start();
  }
}
```


Threads em Java

- Gerenciamento de Threads
  - A classe Thread define diversos métodos para gerenciamento dos threads
    - Métodos estáticos ativados pelo próprio thread
      - Proveem informações sobre o próprio thread
      - Alteram o estado do próprio thread
    - Outros métodos que podem ser invocados por outros threads

```
public static Thread currentThread()
  Retorna uma referência ao thread atualmente em execução.
```

Threads em Java

- Thread em Execução

```
class MyRun implements Runnable {
  public void run() {
    Thread t = Thread.currentThread();
    System.out.println(t.getName() + ": " + t.threadId());
  }
}
```

```
public class RunDemo {
  public static void main(String[] args) {
    MyRun r;
    Thread m, t1, t2;
    r = new MyRun();
    t1 = new Thread(r);
    t2 = new Thread(r);
    m = Thread.currentThread();
    System.out.println(m.getName() + ": " + m.threadId());
    t1.start();
    t2.start();
  }
}
```


Threads em Java

- Interrupções
  - Uma interrupção é uma indicação que o thread deve parar a execução
    - Programador decide como o thread responde a uma interrupção
    - Em geral, o thread termina sua execução
  - Interrupção é gerada invocando o método `interrupt()` do objeto thread a ser interrompido

```
public void interrupt()
```

```
...
t.interrupt();
...
```

Threads em Java

- Interrupções
  - Ao receber interrupção, o thread pode . . .
    - Capturar a exceção `InterruptedException`
      - Somente é possível quando o thread executa métodos que lançam a exceção
    - Verificar se foi interrompido usando o método estático `Thread.interrupted()`

```
public static boolean interrupted()
```

```
...
if (Thread.interrupted()) {
  return;
}
...
```

```
...
if (Thread.interrupted()) {
  throw new InterruptedException();
}
...
```


Threads em Java

- Interrupções

```
class MyRun implements Runnable {
  public void run() {
    Thread t = Thread.currentThread();
    if (Thread.interrupted())
      System.out.println(t.getName() + " interrompido");
  }
}
```

```
public class RunDemo {
  public static void main(String[] args) {
    MyRun r;
    Thread t1, t2;
    r = new MyRun();
    t1 = new Thread(r);
    t2 = new Thread(r);
    t1.start();
    t2.start();
    t1.interrupt();
  }
}
```

Threads em Java

- Interrupções
  - Interrupção é implementada usando o atributo booleano denominado *interrupted*
    - Invocação do método `Thread.interrupt()` liga o atributo *interrupted*
    - Invocação do método `Thread.interrupted()` desliga o atributo *interrupted*
    - Qualquer método que lança a exceção `InterruptedException` desliga o atributo *interrupted*

Threads em Java

- Interrupções
  - Método `Thread.isInterrupted()` pode ser usado por um thread para verificar se outro thread foi interrompido
    - Não altera o atributo *interrupted* do thread verificado
  - É possível ligar/desligar o atributo *interrupted* várias vezes

```
public boolean isInterrupted()
```

Threads em Java

- Pausar Execução
  - Método `Thread.sleep()` suspende a execução do próprio thread por um período de tempo especificado
    - Libera o processador para outros threads
    - Pode ser usado para regular o tempo de execução do thread
    - Precisão depende dos temporizadores do sistema e do escalonador

```
public static void sleep(long millis)
public static void sleep(long millis, int nanos)
```


Threads em Java

- Pausar Execução
  - Pausa pode ser terminada por interrupções
    - Gera a exceção `InterruptedException`

```
...
try {
  Thread.sleep(4000);
} catch (InterruptedException e) {
  return;
}
...
```

Threads em Java

- Pausar Execução
  - Método `Thread.sleep()` suspende a execução do próprio thread por um período de tempo especificado

```
public class HelloWorldSleep extends Thread {
  public void run() {
    System.out.println("Hello World");
  }
   public static void main(String[] args) {
    System.out.println("Iniciando ...");
    HelloWorldSleep t = new HelloWorldSleep();
    t.start();
    sleep(5000);
    System.out.println("Terminando ...");
  }
}
```


Threads em Java

- Pausar Execução
  - Método estático `Thread.yield()` causa o thread pausar temporariamente
    - Libera o processador para outros threads
    - Permite que outros threads possam executar

```
public static void yield()
```

Threads em Java

- Aguardar Término
  - Método `join()` permite um thread aguardar o término de um outro thread
    - Invocação causa o thread pausar até o término do outro thread
  - Também é possível especificar um período de tempo para esperar o término do thread
    - Precisão depende dos temporizadores do sistema e do escalonador
  - Pode ser terminado por interrupções
    - Gera a exceção `InterruptedException`

```
public final void join()
public final void join(long millis)
public final void join(long millis, int nanos)
```


Threads em Java

- Aguardar Término
  - Método `join()` permite um thread aguardar o término de um outro thread

```
public class HelloWorldJoin extends Thread {
  public void run() {
    System.out.println("Hello World");
  }
  public static void main(String[] args) {
    System.out.println("Iniciando ...");
    HelloWorldJoin t = new HelloWorldJoin();
    t.start();
    t.join();
    System.out.println("Terminando ...");
  }
}
```

Threads em Java

- Modos de Execução
  - Non-Daemon
    - Aplicação é finalizada após término de todos threads non-daemon
      - Threads não iniciados não interferem no encerramento da aplicação
    - Thread non-daemon em execução impede encerramento da aplicação
  - Daemon
    - Thread daemon não impede término da aplicação
    - Aplicação pode ser finalizada mesmo com vários threads daemon em execução

Threads em Java

- Modos de Execução
  - Novo thread herda modo do thread pai
    - Thread `main` executa em modo non-daemon
  - Pode ser definido com o método `setDaemon()`
    - Deve ser definido antes de iniciar o thread
    - Em um thread já iniciado, lança a exceção `IllegalThreadStateException`
  - Pode ser avaliado com o método `isDaemon()`

```
public void setDaemon(boolean on)
public boolean isDaemon()
```

```
Thread t = new Thread(r);
t.setDaemon(true);
t.start();
...
if (t.isDaemon()) { ... }
```

Threads em Java

- Modos de Execução

```
class MyRun implements Runnable {
  public void run() {
    Thread t = Thread.currentThread();
    System.out.println(t.getName() + ": " + t.isDaemon());
  }
}
```

```
public class RunDemo {
  public static void main(String[] args) {
    MyRun r = new MyRun();
    Thread t1 = new Thread(r, "T1");
    Thread t2 = new Thread(r, "T2"); t2.setDaemon(true);
    Thread t3 = new Thread(r, "T3"); t3.setDaemon(false);
    Thread m = Thread.currentThread();
    System.out.println(m.getName() + ": " + m.isDaemon());
    t1.start();
    t2.start();
    t3.start();
  }
}
```


Threads em Java

- Métodos Depreciados

```
public final void stop()
  Força o thread parar a execução e terminar
```

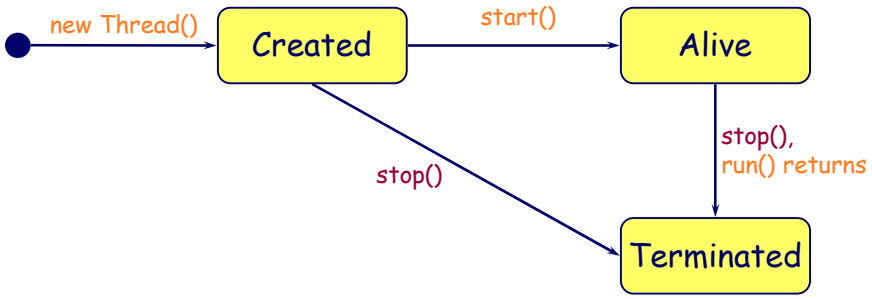
```
public void destroy()
  Destrói o thread (nunca implementado)
```

```
public final void suspend()
  Suspende a execução do thread
```

```
public final void resume()
  Continua a execução do thread suspenso
```

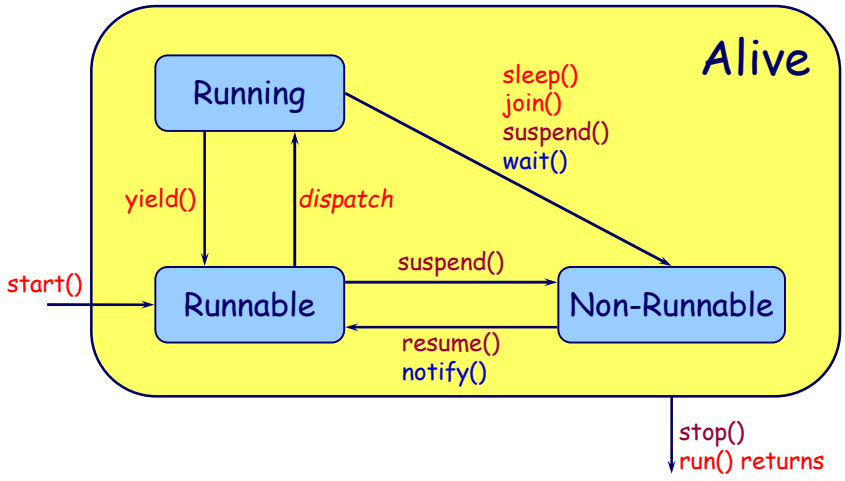
Threads em Java

- Ciclo de Vida



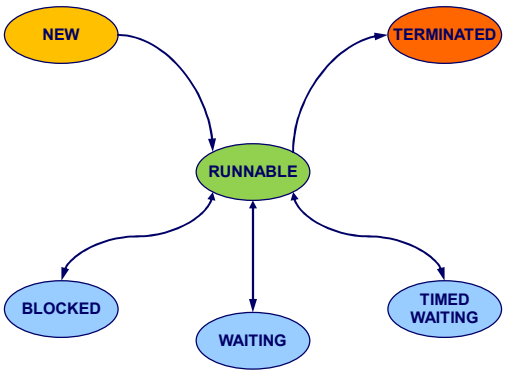
Threads em Java

- Ciclo de Vida



Threads em Java

- Estados Públicos
  - Visíveis externamente pelo método `getState`
  - Definidos pelo tipo enumerado `Thread.State`



Exercício III

2

- Considerando duas matrizes A e B:
  - Implemente um programa sequencial que calcula AxB
    - Mede e exibe o tempo Ts para calcular AxB
    - Exibe a matriz AxB
  - Implemente um programa concorrente que calcula AxB, onde cada elemento de AxB é calculado por um thread
    - Mede e exibe o tempo Tc para calcular AxB
    - Exibe a matriz AxB

Qual a relação entre Ts e Tc?

O desempenho esperado foi observado?

Explique porque o desempenho é ou não é o esperado.

Se não é o esperado, reprojete o uso das threads para obtê-lo.